

【分析】

既然要求找出所有的 x, y , 枚举对象自然就是 x, y 了。可问题在于, 枚举的范围如何? 从 $1/12=1/156+1/13$ 可以看出, x 可以比 y 大很多。难道要无休止地枚举下去? 当然不是。由于 $x \geq y$, 有 $\frac{1}{x} \leq \frac{1}{y}$, 因此 $\frac{1}{k} - \frac{1}{y} \leq \frac{1}{y}$, 即 $y \leq 2k$ 。这样, 只需要在 $2k$ 范围之内枚举 y , 然后根据 y 尝试计算出 x 即可。

7.2 枚举排列

有没有想过如何打印所有排列呢? 输入整数 n , 按字典序从小到大的顺序输出前 n 个数的所有排列。前面讲过, 两个序列的字典序大小关系等价于从头开始第一个不相同位置处的大小关系。例如, $(1, 3, 2) < (2, 1, 3)$, 字典序最小的排列是 $(1, 2, 3, 4, \dots, n)$, 最大的排列是 $(n, n-1, n-2, \dots, 1)$ 。 $n=3$ 时, 所有排列的排序结果是 $(1, 2, 3)$ 、 $(1, 3, 2)$ 、 $(2, 1, 3)$ 、 $(2, 3, 1)$ 、 $(3, 1, 2)$ 、 $(3, 2, 1)$ 。

7.2.1 生成 $1 \sim n$ 的排列

我们尝试用递归的思想解决: 先输出所有以 1 开头的排列 (这一步是递归调用), 然后输出以 2 开头的排列 (又是递归调用), 接着是以 3 开头的排列……最后才是以 n 开头的排列。

以 1 开头的排列的特点是: 第一位是 1, 后面是 $2 \sim 9$ 的排列。根据字典序的定义, 这些 $2 \sim 9$ 的排列也必须按照字典序排列。换句话说, 需要“按照字典序输出 $2 \sim 9$ 的排列”, 不过需注意的是, 在输出时, 每个排列的最前面要加上“1”。这样一来, 所设计的递归函数需要以下参数:

- 已经确定的“前缀”序列, 以便输出。
- 需要进行全排列的元素集合, 以便依次选做第一个元素。

这样可得到一个伪代码:

```
void print_permutation(序列 A, 集合 S)
{
    if(S 为空) 输出序列 A;
    else 按照从小到大的顺序依次考虑 S 的每个元素 v
    {
        print_permutation(在 A 的末尾填加 v 后得到的新序列, S-{v});
    }
}
```

暂时不用考虑序列 A 和集合 S 如何表示, 首先理解一下上面的伪代码。递归边界是 S 为空的情形, 这很好理解: 现在序列 A 就是一个完整的排列, 直接输出即可。接下来按照

从小到大的顺序考虑 S 中的每个元素，每次递归调用以 A 开头。

下面考虑程序实现。不难想到用数组表示序列 A ，而集合 S 根本不用保存，因为它可以由序列 A 完全确定—— A 中没有出现的元素都可以选。C 语言中的函数在接受数组参数时无法得知数组的元素个数，所以需要传一个已经填好的位置个数，或者当前需要确定的元素位置 cur ，代码如下：

```
void print_permutation(int n, int* A, int cur) {
    if(cur == n) {                //递归边界
        for(int i = 0; i < n; i++) printf("%d ", A[i]);
        printf("\n");
    }
    else for(int i = 1; i <= n; i++) {    //尝试在 A[cur] 中填各种整数 i
        int ok = 1;
        for(int j = 0; j < cur; j++)
            if(A[j] == i) ok = 0;        //如果 i 已经在 A[0]~A[cur-1] 出现过，则不能再选
        if(ok) {
            A[cur] = i;
            print_permutation(n, A, cur+1); //递归调用
        }
    }
}
```

循环变量 i 是当前考察的 $A[cur]$ 。为了检查元素 i 是否已经用过，上面的程序用到了一个标志变量 ok ，初始值为 1（真），如果发现有某个 $A[j]=i$ 时，则改为 0（假）。如果最终 ok 仍为 1，则说明 i 没有在序列中出现过，把它添加到序列末尾（ $A[cur]=i$ ）后递归调用。

声明一个足够大的数组 A ，然后调用 $print_permutation(n, A, 0)$ ，即可按字典序输出 $1\sim n$ 的所有排列。

7.2.2 生成可重集的排列

如果把问题改成：输入数组 P ，并按字典序输出数组 A 各元素的所有全排列，则需要对上述程序进行修改——把 P 加到 $print_permutation$ 的参数列表中，然后把代码中的 $if(A[j] == i)$ 和 $A[cur] = i$ 分别改成 $if(A[j] == P[i])$ 和 $A[cur] = P[i]$ 。这样，只要把 P 的所有元素按从小到大的顺序排序，然后调用 $print_permutation(n, P, A, 0)$ 即可。

这个方法看上去不错，可惜有一个小问题：输入 1 1 1 后，程序什么也不输出（正确答案应该是唯一的全排列 1 1 1），原因在于，这样禁止 A 数组中出现重复，而在 P 中本来就有重复元素时，这个“禁令”是错误的。

一个解决方法是统计 $A[0]\sim A[cur-1]$ 中 $P[i]$ 的出现次数 $c1$ ，以及 P 数组中 $P[i]$ 的出现次数 $c2$ 。只要 $c1 < c2$ ，就能递归调用。

```
else for(int i = 0; i < n; i++) {
    int c1 = 0, c2 = 0;
```

```

for(int j = 0; j < cur; j++) if(A[j] == P[i]) c1++;
for(int j = 0; j < n; j++) if(P[i] == P[j]) c2++;
if(c1 < c2) {
    A[cur] = P[i];
    print_permutation(n, P, A, cur+1);
}
}

```

结果又如何呢？输入 1 1 1，输出了 27 个 1 1 1。遗漏没有了，但是出现了重复：先试着把第 1 个 1 作为开头，递归调用结束后再尝试用第 2 个 1 作为开头，递归调用结束后再尝试用第 3 个 1 作为开头，再一次递归调用。可实际上这 3 个 1 是相同的，应只递归 1 次，而不是 3 次。

换句话说，我们枚举的下标 i 应不重复、不遗漏地取遍所有 $P[i]$ 值。由于 P 数组已经排过序，所以只需检查 P 的第一个元素和所有“与前一个元素不相同”的元素，即只需在“for($i = 0; i < n; i++$)”和其后的花括号之前加上“if($i || P[i] != P[i-1]$)”即可。

至此，结果终于正确了。

7.2.3 解答树

假设 $n=4$ ，序列为 $\{1,2,3,4\}$ ，如图 7-1 所示的树显示出了递归函数的调用过程。其中，结点内部的序列表示 A ，位置 cur 用高亮表示，另外，由于从该处开始的元素和算法无关，因此用星号表示。

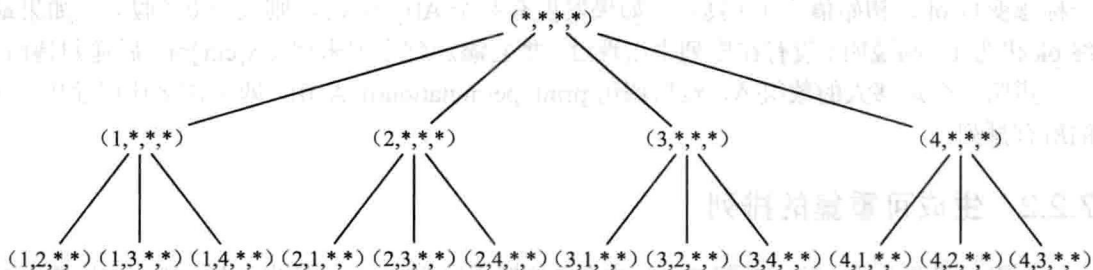


图 7-1 排列生成算法的解答树

这棵树和前面介绍过的二叉树不同。第 0 层（根）结点有 n 个子结点，第 1 层结点各有 $n-1$ 个子结点，第 2 层结点各有 $n-2$ 个子结点，第 3 层结点各有 $n-3$ 个子结点，……，第 n 层结点都没有子结点（即都是叶子），而每个叶子对应于一个排列，共有 $n!$ 个叶子。由于这棵树展示的是从“什么都没做”逐步生成完整解的过程，因此将其称为解答树。

提示 7-2：如果某问题的解可以由多个步骤得到，而每个步骤都有若干种选择（这些候选方案集可能会依赖于先前作出的选择），且可以用递归枚举法实现，则它的工作方式可以用解答树来描述。

这棵解答树一共有多少个结点呢？可以逐层查看：第 0 层有 1 个结点，第 1 层 n 个，第

2层有 $n*(n-1)$ 个结点（因为第1层的每个结点都有 $n-1$ 个结点），第3层有 $n*(n-1)*(n-2)$ 个（因为第2层的每个结点都有 $n-2$ 个结点），……，第 n 层有 $n*(n-1)*(n-2)*\dots*2*1=n!$ 个。

下面把它们加起来。为了推导方便，把 $n*(n-1)*(n-2)*\dots*(n-k)$ 写成 $n!/(n-k-1)!$ ，则所有结点之和为：

$$T(n) = \sum_{k=0}^{n-1} \frac{n!}{(n-k-1)!} = n! \sum_{k=0}^{n-1} \frac{1}{(n-k-1)!} = n! \sum_{k=0}^{n-1} \frac{1}{k!}$$

根据高等数学中的泰勒展开公式， $\lim_{n \rightarrow \infty} \sum_{k=0}^{n-1} \frac{1}{k!} = e$ ，因此 $T(n) < n! e = O(n!)$ 。由于叶子有 $n!$ 个，倒数第二层也有 $n!$ 个结点，因此上面的各层全部加起来也不到 $n!$ 。这是一个很重要的结论：在多数情况下，解答树上的结点几乎全部来源于最后一两层。和它们相比，上面的结点数可以忽略不计。

不熟悉泰勒展开公式也没有关系：可以写一个程序，输出 $\sum_{k=0}^{n-1} \frac{1}{k!}$ 随着 n 增大时的变化，并发现它能很快收敛。这就是计算机的优点之一——可以通过模拟避开数学推导。即使无法严密而精确地求解，也可以找到令人信服的实验数据。

7.2.4 下一个排列

枚举所有排列的另一个方法是从字典序最小排列开始，不停调用“求下一个排列”的过程。如何求下一个排列呢？C++的 STL 中提供了一个库函数 `next_permutation`。看看下面的代码片段，就会明白如何使用它了。

```
#include<cstdio>
#include<algorithm> //包含 next_permutation
using namespace std;
int main() {
    int n, p[10];
    scanf("%d", &n);
    for(int i = 0; i < n; i++) scanf("%d", &p[i]);
    sort(p, p+n); //排序，得到 p 的最小排列
    do {
        for(int i = 0; i < n; i++) printf("%d ", p[i]); //输出排列 p
        printf("\n");
    } while(next_permutation(p, p+n)); //求下一个排列
    return 0;
}
```

需要注意的是，上述代码同样适用于可重集。

提示 7-3：枚举排列的常见方法有两种：一是递归枚举，二是用 STL 中的 `next_permutation`。